Smart-X IoT Mesh Ecosystem

Project Completion Report and Technology Recommendations (Final PoE)

Student Muaaz Abdool-Gaffoor (ST10443760)
Modules PROG7312 Programming 3B / AAPD7112 Advanced Application Development
Source code https://github.com/EMKNDW/prog7312-g2-2026-prog7312-poe-st10443760
Part 3 video https://youtu.be/QKaKJ5zSwYA

1. Overview of the completed project

Smart-X simulates an IoT ecosystem in which ESP32-style nodes (soil moisture sensors, smart power meters, pumps, valves and relays) send telemetry to a central gateway. The finished system is a .NET 10 Minimal API with a SignalR hub and background workers, a Blazor WebAssembly dashboard, a shared class library that holds the data structures and models, a console simulator that behaves like a fleet of real devices, and an xUnit test project. Everything runs locally with *dotnet run* or in Docker with a single *docker compose up --build* (API on port 8080, client on port 8081, simulator as an optional profile).

Part	What it added	Main data structures and algorithms
1. Ingestion and Telemetry	Sensor registration with a nested deployment location, file attachments, live Mesh Pulse widget, validation.	Generic *TelemetryPacket<T>*, operator overloading, jagged array to *List<T>*, recursive location validator
2. Real-Time Command Stream	Live dashboard over SignalR, manual overrides with undo, search and filters, alert handling, an action engine that suggests the next step.	Queue, PriorityQueue, Stack, Dictionary, SortedDictionary, HashSet, transition-frequency model
3. Network Topology and Mesh Routing	Mesh map, per-sensor routes, live self-healing view, historical log with a time range picker.	AVL tree with range queries, graph (adjacency list), breadth-first search, Prim's algorithm with a heap

The repository holds more than 200 commits across the three parts. The test project has 146 automated tests (90 from Parts 1 and 2 and the rest for Part 3, including the fixes made after the first build), and a fresh clone builds with no warnings or errors.

2. How Task 3 was completed

Task 3 was built in six batches. First came the AVL tree and a per-device history store, because every history question in the feature depends on it. Next came the graph model and the seeded mesh topology, then breadth-first search for reachability, then Prim's algorithm and route lookup. After the algorithms were tested on their own, the topology page was built on top of them, followed by the historical log, and finally Docker, the README, the implementation report and the feedback file.

The key design decision was to keep every algorithm in its own small class in the shared library, with the API endpoints kept thin and the Blazor components only drawing results. That made each piece testable without a browser, and it let the topology page be rebuilt from one function whenever a node changed state.

3. Challenges and how they were overcome

3.1 Algorithmic challenges

Challenge	What happened and how it was solved
A node that is up can still be unreachable	When both neighbours of Moisture Sensor C are down, the sensor itself is healthy but has no path to the gateway. The first idea (treat "down" as the only bad state) hid this. Breadth-first search now starts at the gateway, skips down nodes, and classifies every node it never reaches as either **Down** or **Cut off**. The map shows the two differently so an operator can tell a dead node from an isolated one.
Prim's algorithm on a split network	Textbook Prim's assumes a connected graph. With two nodes down the network can split into islands. The implementation restarts from the first unvisited node when the heap runs dry, so it returns a spanning forest, reports the nodes that cannot reach the gateway, and never loops or throws.
MST routes are not shortest paths	Valve 1's route through the minimum spanning tree costs 14.8, while its direct link to the gateway costs 11.1. This looked like a bug at first. It is correct: the tree minimises the total power cost of the whole network, not each node's own route. This is explained in the README and shown in the demo, because it is the main difference between an MST and shortest-path routing.
Wrong "where traffic hops	The reroute report listed the gateway itself as a neighbouring node a down device could hop

instead" result	through. A unit test for the reroute rule caught it, and the gateway is now excluded from that list.
Keeping a range query efficient	The AVL tree needed pruning, not just an in-order walk. The range query only enters the left subtree if the start of the range is below the node and the right subtree if the end is above it. The page shows the result: a one-hour query for Moisture Sensor A matched 685 readings and visited 691 of 2,880 stored, and narrower ranges visit a smaller share.

3.2 Architecture and integration challenges

Challenge	What happened and how it was solved
Making the topology live	The disconnect detector and alert set from Part 2 already knew when a node changed state, but nothing else reacted. The alert broadcaster now also pushes a freshly computed topology over a new SignalR event. The client ignores a snapshot older than the one it holds, so a REST response and a push that cross on the wire cannot roll the map back.
Demonstrating self-healing from the dashboard	Part 2 could only disconnect a device, so a seeded device stayed down forever without the console simulator. A reconnect path was added that also tells the detector to stop watching the node until it really reports again; without that second part the detector flagged the device again ten seconds later. With the simulator running, a button disconnect only lasts until the device's next packet (about two seconds), so the README and the demo explain how to silence the node from the simulator instead.
Date picker that kept resetting	On the history card the From and To times reset as soon as the user clicked away. The first theory was that SignalR re-renders were overwriting the input. Logging every write to the input, with a stack trace, showed this was wrong: Blazor passes a *datetime-local* value with seconds added, and the bound format only allowed minutes, so the parse failed every time and Blazor reverted the box. The fix was to edit draft values and commit them on Run query, with a helper that accepts both formats and a regression test. The lesson was to test the theory with evidence before changing code.
Razor and SVG	SVG *text* elements inside *@if* blocks were read by Razor as its own text tag and broke the build. Wrapping them in group elements fixed it. The map also shrank to about 65% on the page, so labels became unreadable and two nodes shared the initials "PA". The sizes were enlarged and the initials rule changed so every node is unique.
Number formatting	A route message read "6,5" instead of "6.5" because it used the server's regional number format. All numbers shown to users now use invariant formatting.
An intermittent build error	The client build failed four times with an *ApplyCompressionNegotiation* duplicate-key error, and the next build passed each time. A fresh clone never showed it. The cause was not pinned down. It is noted here rather than hidden, and the final check was a clean build from a fresh clone.

3.3 Environment and tooling challenges

Challenge	What happened and how it was solved
Docker serving Blazor correctly	In Part 1 the client container downloaded a file instead of showing the site, because nginx did not serve *.wasm* with the right type and had no fallback to *index.html*. The attachment links were also hardcoded to a development port. Both were fixed in the nginx configuration and the API base address, which is why Part 3 runs in Docker without extra steps.
Losing the development laptop during Task 3	The laptop stopped working part-way through. The project was cloned onto another computer and the tools reinstalled: Git, the .NET 10 SDK, and Docker Desktop. Docker needed WSL 2 to be enabled with administrator rights, which blocked the first install. Because the code was pushed after every batch, no work was lost.
Git identity and two remotes	On the new computer a placeholder was pasted into the Git settings, so 47 commits were authored under the wrong name. Before anything reached the marked repository, the author was corrected on the working repository only, with a lease-protected push, and the marked repository is only ever updated with normal pushes.

4. Key learnings

- **Choose the structure for the question being asked.** The same assignment used a queue for fairness, a priority queue and a heap for "most urgent first" and "cheapest link first", a stack for undo, a dictionary for instant lookup, a hash set for "is this already flagged", a tree for ranges, and a graph for connections. In this project the AVL tree answered a one-hour history query by visiting 691 of 2,880 readings, which made the benefit visible instead of theoretical.
- **Algorithms meet real data.** Prim's and breadth-first search were easy to describe and harder to make robust: disconnected graphs, nodes that are up but isolated, and routes that are not shortest paths all needed explicit handling and tests.
- **Shared state needs care.** Part 2 introduced a worker and endpoints that touch the same structures, so each

structure has its own lock. A Part 1 collection that was fine on its own had to be changed once the simulator was registering devices while other code was reading it. The feedback file records that change.

- **Tests earn their place.** The reroute bug and the date format bug are both covered by tests that would fail if the problem came back. 146 tests make it safe to change a shared algorithm.
- **Debug with evidence.** The date picker taught that a plausible theory can be wrong. Logging what actually wrote to the input gave a faster and more certain answer than guessing.
- **State has to survive navigation.** A singleton state container in the client, the same pattern across all three pages, keeps filters, selections, feeds and ranges when the user moves around.
- **Deployment is part of the feature.** Most of the Docker problems (MIME types, ports, WSL) had nothing to do with the C# code, but they decided whether the app ran at all.
- **Skills gained:** .NET 10 Minimal APIs, Blazor WebAssembly with SignalR, writing and testing an AVL tree and graph algorithms, Docker Compose with nginx, and working in small, frequent commits.

5. Use of AI tools

Claude and Claude Code were used mainly for planning and for debugging across the three parts, to help set up the initial project structure, and to help draft and format this report and the demo scripts. The full statement is in the code attribution document in the repository.

6. Technology recommendations

The current system keeps all data in memory, receives telemetry over HTTP from a simulator, and runs as a single instance. The recommendations below remove those limits, in the order they would be worth doing. Each one works with the existing .NET code.

Technology	What it adds to Smart-X	Why it suits this project and .NET
GitHub Actions CI/CD	Builds the solution, runs the 146 tests and builds the Docker images on every push, and can publish the images to a registry.	The repository is already on GitHub and the solution builds with *dotnet build* and *dotnet test*, so the workflow is a few steps. It would have caught the intermittent build error and any broken commit before it reached the marked branch.
Persistent time-series storage (Azure SQL with EF Core, or PostgreSQL)	History survives restarts and is not limited by memory. The AVL tree becomes a fast in-memory cache over recent readings, with older data queried from the database.	Entity Framework Core is the standard .NET data layer. The history capacity per device is currently a fixed limit, so a database is the natural next step. It also removes the need to re-seed history on every start.
MQTT (MQTTnet) or Azure IoT Hub for ingestion	Real ESP32 devices could publish telemetry directly instead of posting to an HTTP endpoint. MQTT is lightweight and keeps working on unreliable wireless links.	MQTTnet is a .NET library for MQTT clients and brokers, and Azure IoT Hub supports MQTT from devices. The existing ingestion pipeline (queue, priority queue, worker) can stay as it is, with the MQTT subscriber replacing the HTTP endpoint as the producer.
Azure SignalR Service	Lets the dashboard scale beyond one API instance, because connections are handled by a managed service.	It is designed for ASP.NET Core SignalR and normally needs only a registration change, so the hub and the Blazor client code stay the same. It matters once more than one API container runs.
OpenTelemetry with Application Insights	Tracing, metrics and logs for the queue depth, worker delay and alert counts that the dashboard already shows to users.	.NET has built-in support for OpenTelemetry, and the same data can feed alerts for the operators of the platform, not only its users.
An LLM summary of daily alerts (Microsoft.Extensions.AI)	A short written summary of the day's alerts, disconnects and suggested actions for an operator, in plain language.	Microsoft.Extensions.AI gives .NET a common interface to language model services, so the provider can be swapped. The summary would be generated from structured data the system already holds, and the model would only phrase it. Outputs should be labelled as generated and never trigger commands.
Home Assistant integration	Shows Smart-X devices and alerts in a popular open-source smart home dashboard.	Home Assistant can read devices published over MQTT, so the MQTT recommendation above would also unlock it with little extra code.

> **Security note for all of the above:** moving from a simulator to real devices means adding authentication (for example ASP.NET Core Identity or Microsoft Entra ID), per-device credentials and HTTPS. These are available in the same .NET ecosystem.

7. Conclusion

All three pillars of Smart-X now work together in one menu, in one Docker Compose stack, with the data structures from Part 2 feeding the tree and graph from Part 3. The hardest parts were not the algorithms themselves but making them behave on a changing network and keeping the live page in step with it. The recommendations in section 6 are the path from a simulated demonstrator to a system that could run real devices.

References

Adelson-Velsky, G.M. and Landis, E.M. (1962) An algorithm for the organization of information. *Proceedings of the USSR Academy of Sciences*, 146, pp. 263-266.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*. 3rd edn. Cambridge, MA: MIT Press.

GitHub (2026) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions

Home Assistant (2026) *Home Assistant*. Available at: https://www.home-assistant.io/

Microsoft (2026) *ASP.NET Core SignalR*. Available at: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction

Microsoft (2026) *Azure IoT Hub documentation*. Available at: https://learn.microsoft.com/en-us/azure/iot-hub/

Microsoft (2026) *Azure SignalR Service documentation*. Available at: https://learn.microsoft.com/en-us/azure/azure-signalr/

Microsoft (2026) *Entity Framework Core*. Available at: https://learn.microsoft.com/en-us/ef/core/

Microsoft (2026) *Microsoft.Extensions.AI libraries*. Available at: https://learn.microsoft.com/en-us/dotnet/ai/microsoft-extensions-ai

Microsoft (2026) *Observability with OpenTelemetry in .NET*. Available at: https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel

Microsoft (2026) *PriorityQueue<TElement,TPriority> class*. Available at: https://learn.microsoft.com/dotnet/api/system.collections.generic.priorityqueue-2

MQTTnet (2026) *MQTTnet: a .NET library for MQTT*. Available at: https://github.com/dotnet/MQTTnet